NODEBASE - INTERVIEW PREPARATION

Challenges and Design Decisions

Real bugs, debugging methods, failure handling, scaling, security and production readiness.

8 questions, answered in simple language

Each answer is written to be spoken in an interview. It is based on the actual codebase: Next.js 15, React 19, TypeScript, tRPC, Prisma, PostgreSQL, Better Auth, Polar, Inngest, BullMQ, Redis and React Flow.

Prepared in 2026

1. Describe a bug that took you a long time to fix.

The bug that took me the longest was the email OTP flow. Sign-up returned a successful response, but no email arrived. At first it looked like a database or auth problem, because the user row was created and the verification row existed. The real issue was a chain of separate failures, which is why it took so long to find.

The first problem was the trusted origin configuration. The application compared requests against the ngrok URL, so opening the app on localhost produced an invalid origin error. That error returned a 403, and the login form treated every 403 as an unverified email, so the user was silently redirected to the verification page. That hid the real cause. I fixed it by adding the local origins and normalising trailing slashes.

The second problem was the SMTP credentials. The worker was running, but Gmail rejected the login with a 535 error because the password was not a valid App Password. I found this by inspecting the BullMQ failed job in Redis and reading its `failedReason`, not by guessing from the UI. After switching to a 16-character App Password and aligning the sender address with the authenticated account, email delivery worked.

The third problem was environmental. The SMTP transport is created once when the module loads, so editing `.env` required restarting the worker. There was also a Windows-specific `mprocs` error because it resolved `npm` to a non-executable shell script; changing the commands to `npm.cmd` fixed it. The lesson was to verify each hop independently and to trust the actual queue and database state over the UI.

2. What technical decision would you change today?

The decision I would change is how a node references a credential. The Prisma schema declares a `credentialId` column on the Node table with a foreign key to Credential, but the application never writes that column. Instead, the credential id is stored inside the node's JSON `data` object. That means there are two mechanisms for one relationship, and the real one has no database-level integrity.

The consequence is that deleting a credential does not clear the references in node data. A workflow can keep a credential id that points to a row that no longer exists, and the failure only appears at execution time as a credential not found error. The database cannot prevent a node from referencing another user's credential either, although the executors do scope the lookup by user id at runtime, so the security consequence is limited.

Today I would choose one approach and make it consistent. The cleaner option is to use the real foreign key column, set it when the node dialog is saved, and remove the credential id from the JSON data. That would give referential integrity, make it easy to find which workflows use a credential, and allow a controlled delete or set-null behaviour. The alternative is to remove the unused column and document the JSON convention, but then the application must explicitly clean up references when a credential is deleted.

I would also revisit the empty-data rule in `workflows.update`. Ignoring empty data protects against wiping configuration from an incomplete snapshot, but it makes it impossible to clear a node through the UI. A clearer contract would distinguish an absent data field from an explicitly empty object.

3. How did you debug database-related problems?

I start by reproducing the problem with the smallest possible case and identifying the exact query that fails. Prisma errors usually include a code and a message, such as a unique constraint violation or a missing record. I read the error carefully before changing anything, because the first message often points to the real cause. For example, a `findUniqueOrThrow` failure during save usually means the workflow id or the user id did not match.

Then I check the schema and the actual database, not just the TypeScript code. I use Prisma queries and PostgreSQL catalog queries to inspect columns, foreign keys, indexes and constraints. That is how I confirmed that the unique index on `Execution.inngestEventId` exists even though Prisma implements it as an index rather than a named constraint. I also compare the live rows with what the application expects, especially for JSON fields and nullable columns.

For transaction problems, I look at the order of operations. Saving a workflow deletes removed nodes, replaces connections and touches the workflow row. If a connection references a node that was just deleted, the cascade may remove it, and if a new connection references a node that was not created, the foreign key will fail. I isolate the failing step by logging the inputs or by running the queries in a transaction that I can roll back.

For performance or locking issues, I use `EXPLAIN` and check indexes, and I pay attention to connection limits. The workflow creation limit uses a row lock, so I also think about contention there. Finally, I check the environment: the wrong database URL, a missing migration, or a pooled connection with different behaviour can all look like application bugs.

4. How did you handle failures in background jobs?

There are two background systems, and they handle failure differently. Workflow execution runs in Inngest. The function has zero retries in development and three in production. Each long operation is wrapped in `step.run`, so a retry replays the function but returns memoized results for steps that already succeeded. That prevents duplicate execution rows and repeated side effects from earlier nodes. Configuration errors use `NonRetriableError`, so they fail immediately instead of wasting the retry budget.

When retries are exhausted, the `onFailure` hook marks the execution FAILED and stores the error message and stack trace. That row is linked to the run through the unique Inngest event id. This gives the user and the developer a durable record of what happened, even if the function failed outside its own final update step.

Email and cron jobs run in BullMQ. The email queue uses three attempts with exponential backoff, removes successful jobs immediately, and keeps up to a thousand failed jobs for inspection. The cron queue uses the same retry idea with a longer backoff. The workers attach failed listeners that log the job id and the reason, and they handle SIGTERM and SIGINT by closing gracefully so in-flight jobs can finish. BullMQ also has a stalled-job checker that requeues work if a worker dies.

There are still gaps. Delivery is at-least-once, so a retry can send a duplicate email. Failed executions do not set `completedAt`, so failure duration cannot be calculated. Queue depth is not monitored, and the health endpoint does not check the database or the workers. In production I would add alerts, a dead-letter review process, and a small support window where completed email jobs are retained.

5. What happens if the database becomes unavailable?

If PostgreSQL becomes unavailable, most interactive requests fail. tRPC procedures call Prisma, and Prisma will throw a connection error. The client receives a server error, and depending on the page, the error boundary shows an error view or the mutation shows a toast. Authentication also fails because Better Auth uses the same database for sessions, so users may be unable to sign in or verify their email.

Workflow execution is affected in a more subtle way. Inngest can still deliver the event, but the `create-execution` step will fail because it cannot insert the execution row. In the current code, that error is wrapped in a `NonRetriableError`, which is a deliberate trade-off: it avoids burning retries on a deterministic problem. The downside is that a genuinely transient database outage is not retried at that step. Later steps that read the workflow would also fail, and the `onFailure` hook could fail as well because it also needs the database.

Background workers are similarly affected. The email worker does not need PostgreSQL to send an email, because the OTP data is already in the job, but the auth request that created the job would have failed earlier. The cron worker can send events to Inngest, but the execution function cannot create the execution row. Rate limiting and idempotency may still work if Upstash is available, which helps prevent duplicate webhooks during the outage.

To make the system more resilient, I would add connection pooling and timeouts, retry transient connection errors with backoff, configure Neon high availability and backups, and add a circuit breaker so the application fails fast instead of hanging. I would also make the health check verify the database and queue connectivity, and add a user-facing maintenance state for planned outages.

6. How would you support 10,000 concurrent users?

The first step is to separate the interactive path from the heavy work, which the architecture already does. Next.js handles page requests and tRPC calls, while Inngest runs workflows and BullMQ runs email and cron jobs. To support ten thousand concurrent users, I would run multiple stateless Next.js instances behind a load balancer and scale them horizontally based on CPU, memory and request rate.

Database capacity would be the next bottleneck. I would use a proper connection pooler such as PgBouncer or the Neon pooler, cap the number of connections per instance, and add indexes for the common queries. The workflow and credential lists are already paginated. Execution history would eventually need partitioning or archiving because it grows quickly. I would also consider read replicas for reporting and non-critical reads, while keeping writes on the primary.

Caching and rate limiting would reduce repeated load. Static assets would go through a CDN, and frequently read data such as subscription state or workflow metadata could be cached briefly in Redis. Upstash already handles rate limits, but I would make the limits stricter and fail closed for public webhooks. For realtime, I would reduce the number of subscriptions: today each node card opens its own subscription, so a large workflow creates many connections. I would subscribe once per workflow or per run and distribute the messages in the client.

Finally, I would add observability and load testing. I would measure p95 latency, database connection usage, queue depth, Inngest concurrency and error rates, then run load tests against the real bottlenecks. Inngest and BullMQ should have concurrency limits so a traffic spike does not overwhelm external APIs. Autoscaling, backpressure and clear alerts are as important as raw server capacity.

7. How would you make the application more secure?

The first area is authentication and authorization. The procedure chain already re-checks the session on the server, and ownership is included in every workflow, credential and execution query. I would keep that discipline and add tests for it. I would also add a unique index on `Account(providerId, accountId)`, encrypt OAuth tokens or stop requesting unnecessary scopes, and make session cookie settings explicit for production.

The second area is webhook and API security. The Google Form and Stripe webhook routes are public, so they rely on rate limiting and idempotency rather than authentication. The Telegram route verifies a secret token with a timing-safe comparison, which is the right model. For Google Form and Stripe I would add a shared secret or signature verification and make the rate limit fail closed when Redis is not configured. I would also restrict the HTTP node, because a user could otherwise make the server call internal addresses such as cloud metadata endpoints. That needs an allowlist or a block on private IP ranges to prevent SSRF.

The third area is secrets and data protection. Credentials are encrypted with AES-256-GCM through Cryptr, but the encryption key has no rotation path. I would move secrets to a managed secret store, add key rotation and re-encryption, and make sure logs never contain plaintext keys or tokens. I would add security headers, a content security policy, dependency scanning and secret scanning in CI. Sentry should be configured to scrub sensitive values.

Finally, I would add audit logging for sensitive actions such as credential access, workflow deletion and billing changes, and I would review the queue dashboard authentication. Security is not one feature; it is a set of defaults and verification steps that must be maintained as the product grows.

8. What would you change before using this project in production?

Before production I would fix the known correctness bugs first. I would make manual executions use the shared dispatch helper so they get an idempotency key. I would set `completedAt` on failure. I would validate cycles when a workflow is saved instead of only at execution time. I would fix the node selector branch that can wipe a canvas when an `INITIAL` node exists, and I would fix the OpenAI executor that publishes to the Anthropic channel. I would also resolve the unused `Node.credentialId` column.

Next I would harden security and operations. I would add webhook authentication for public endpoints, make rate limiting fail closed, protect the HTTP node against SSRF, rotate the encryption key, encrypt OAuth tokens, and move secrets into a managed store. I would add security headers, dependency scanning and log scrubbing. I would also configure backups and point-in-time recovery for PostgreSQL, set up alerting for failed executions and queue depth, and add structured logs and Sentry alerts.

Then I would improve the engineering process. I would add unit, integration and end-to-end tests, and run them in CI together with lint, typecheck and build. I would automate database migrations and container builds, add staging and production environments, and write runbooks for common failures. I would load test the application and set resource limits and autoscaling policies.

Finally, I would finish the deployment work. The EKS path needs Kubernetes manifests or Helm charts, secret management, health and readiness probes, worker deployments, and verified runtime environment handling. I would also add team workspaces, roles and audit logs before selling to larger customers. Until those are done, I would treat the project as a strong portfolio and prototype system, not as a fully production-ready SaaS.